COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010-CPE21S4	**SECTION:**	CPE21S4
DATE PERFORMED:	19/10/2026	**DATE SUBMITTED:**	19//010/2026
NAME:	Jullian Edrain R. Sevilla/ Angel Mae Ramirez	**INSTRUCTOR:**	Engr. JIMLORD QUEJADO

ACTIVITY NO. ___ [ACTIVITY NAME]

1. PROCEDURE OUTPUT

What procedures did you perform?

CPE010_HOA3p1_SevillaRemirez_sc1.cpp CPE010_HOA3p1_SevillaRemirez_sc2.cpp

CPE010_HOA3p1_SevillaRemirez_sc1.cpp > ...

```cpp
#include <iostream>
#include <thread>
#include <string>

void print(int n, const std::string &str) {
    std::cout << "Printing integer: " << n << std::endl;
    std::cout << "Printing string: " << str << std::endl;
}

int main() {
    std::thread t1(print, 10, "T.I.P.");
    t1.join();

    return 0;
}
```

CPE010_HOA3p1_SevillaRemirez_sc1.cpp | CPE010_HOA3p1_SevillaRemirez_sc2.cpp

CPE010_HOA3p1_SevillaRemirez_sc2.cpp > ...

```cpp
#include <iostream>
#include <thread>
#include <string>
#include <vector>

void print(int n, const std::string &str) {
    std::string msg = std::to_string(n) + ":" + str;
    std::cout << msg << std::endl;
}

int main() {
    std::vector<std::string> s = {
        "T.I.P.",
        "Competent",
        "Computer",
        "Engineers"
    };

    std::vector<std::thread> threads;

    for (int i = 0; i < s.size(); i++) {
        threads.push_back(std::thread(print, i, s[i]));
    }

    for (auto &th : threads) {
        th.join();
    }

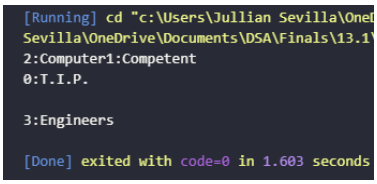
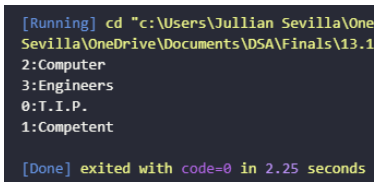
    return 0;
}
```

Table 13.1 Simple One-Threaded Example

Screenshot	[Running] cd "c:\Users\Jullian Sevilla\OneDr Sevilla\OneDrive\Documents\DSA\Finals\13.1\" Printing integer: 10 Printing string: T.I.P. [Done] exited with code=0 in 1.513 seconds	[Running] cd "c:\Users\Jullian Sevilla\OneD Sevilla\OneDrive\Documents\DSA\Finals\13.1\ Printing integer: 10 Printing string: T.I.P. [Done] exited with code=0 in 3.985 seconds

Analysis	Sevilla - Both runs print the same words because a single thread always runs in order. The t1.join() command forces the program to wait until the thread finishes before closing. The second run took longer (3.98s vs 1.51s) simply because the computer was busy running other background tasks.	

Table 13.2 Multithreaded Example.

Screenshot		
Analysis	Sevilla - The two runs give different results because four threads run at the same time without waiting for each other. In the first run, two threads printed at the exact same time, causing the text to get messy and mix together. In the second run, the threads printed in a totally different order. This proves that threads finish in random order every time you run the code because there is no lock.	

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

Sevilla

1. Write a definition of multithreading and its advantages/disadvantages.
    - Multithreading means letting a single program run multiple small tasks, called threads, at the same time. The main advantage is that it speeds up programs by using all CPU cores instead of just one. It also keeps apps from freezing during heavy work, and it makes sharing data between threads very fast. The main disadvantage is that threads can mess each other up if they try to change the same data at the same time without waiting their turn. Also, making too many threads uses up extra memory, and finding bugs in thread code is hard because tasks finish in random order.
2. Rationalize the use of multithreading by providing at least 3 use-cases.
    - We use multithreading so a program does not get stuck waiting for one slow job to finish before starting another. One example is video games, where one thread draws the graphics while another thread calculates physics and enemy movement so the screen stays smooth. Another example is web servers, where worker threads answer thousands of website visitors at the same time. A third example is sorting huge lists of data, where threads split a large pile of numbers into smaller pieces so all CPU cores can sort them much faster.
3. Differentiate between parallelism and concurrency.
    - Concurrency is about juggling multiple tasks, so they overlap in time, while parallelism is about actually doing multiple tasks at the same second on separate CPU cores. Concurrency can happen on a computer with only one CPU core by quickly switching back and forth between tasks. Parallelism needs a computer with two or more physical cores to run tasks together. Concurrency is mainly used to keep programs responsive, while parallelism is used to make big calculations finish faster. In simple terms, concurrency is like one worker taking turns to serve two lines of people, while parallelism is having two workers serving two lines at the exact same time.

Ramirez

1. Write a definition of multithreading and its advantages/disadvantages.
    - ans
2. Rationalize the use of multithreading by providing at least 3 use-cases.
    - ans
3. Differentiate between parallelism and concurrency.
    - ans

Supplementary part B

CPE010_HOA3p1_SevillaRemirez_sc3.cpp.cpp

supplementary > CPE010_HOA3p1_SevillaRemirez_sc3.cpp.cpp > ...

```cpp
#include <iostream>
#include <thread>

int global_var = 0;

void add(int value) {
    global_var += value;
}

int main() {
    std::cout << "Initial global variable value: " << global_var << std::endl;

    std::thread t1(add, 10);
    std::thread t2(add, 20);
    std::thread t3(add, 30);

    t1.join();
    std::cout << "After T1.join(), global variable value: " << global_var << std::endl;

    t2.join();
    std::cout << "After T2.join(), global variable value: " << global_var << std::endl;

    t3.join();
    std::cout << "After T3.join(), global variable value: " << global_var << std::endl;

    return 0;
}

```

```
[Running] cd "c:\Users\Jullian Sevilla\OneD
cpp && "c:\Users\Jullian Sevilla\OneDrive\D
Initial global variable value: 0
After T1.join(), global variable value: 30
After T2.join(), global variable value: 60
After T3.join(), global variable value: 60

[Done] exited with code=0 in 1.384 seconds
```

Analysis

The global variable starts at zero before any threads run. After T1 joins, the value becomes 30 because T1 and T2 completed their execution. After T2 and T3 join, the final value reaches 60. Modifying shared global memory without locks creates a race condition that can corrupt data in larger programs.

Part C

CPE010_HOA3p1_SevillaRemirez_sc4.cpp

supplementary > CPE010_HOA3p1_SevillaRemirez_sc4.cpp > ...

```cpp
#include <iostream>
#include <vector>
#include <thread>
#include <chrono>
#include <algorithm>
#include <random>

void merge(std::vector<int>& arr, int left, int mid, int right) {
    std::vector<int> temp(right - left + 1);
    int i = left, j = mid + 1, k = 0;

    while (i <= mid && j <= right) {
        if (arr[i] <= arr[j]) {
            temp[k++] = arr[i++];
        } else {
            temp[k++] = arr[j++];
        }
    }
    while (i <= mid) temp[k++] = arr[i++];
    while (j <= right) temp[k++] = arr[j++];

    for (i = 0; i < k; ++i) {
        arr[left + i] = temp[i];
    }
}

void sequentialMergeSort(std::vector<int>& arr, int left, int right) {
    if (left >= right) return;
    int mid = left + (right - left) / 2;
    sequentialMergeSort(arr, left, mid);
    sequentialMergeSort(arr, mid + 1, right);
    merge(arr, left, mid, right);
}

void parallelMergeSort(std::vector<int>& arr, int left, int right, int depth = 0) {
    if (left >= right) return;

    int mid = left + (right - left) / 2;

    if (depth < 2) {
        std::thread leftThread(parallelMergeSort, std::ref(arr), left, mid, depth + 1);
        parallelMergeSort(arr, mid + 1, right, depth + 1);
        leftThread.join();
    } else {
        sequentialMergeSort(arr, left, mid);
        sequentialMergeSort(arr, mid + 1, right);
    }

```

```cpp
    merge(arr, left, mid, right);
}

int main() {
    const int N = 1000000;
    std::vector<int> arr1(N);
    std::vector<int> arr2(N);

    std::mt19937 rng(42);
    for (int i = 0; i < N; ++i) {
        int val = rng() % 1000000;
        arr1[i] = val;
        arr2[i] = val;
    }

    auto startSeq = std::chrono::high_resolution_clock::now();
    sequentialMergeSort(arr1, 0, N - 1);
    auto endSeq = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double, std::milli> seqDuration = endSeq - startSeq;

    auto startPar = std::chrono::high_resolution_clock::now();
    parallelMergeSort(arr2, 0, N - 1);
    auto endPar = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double, std::milli> parDuration = endPar - startPar;

    std::cout << "--- Parallel Algorithm Benchmark: Merge Sort ---" << std::endl;
    std::cout << "Dataset Size:              " << N << " elements" << std::endl;
    std::cout << "Sequential Merge Sort Time: " << seqDuration.count() << " ms" << std::endl;
    std::cout << "Parallel Merge Sort Time:   " << parDuration.count() << " ms" << std::endl;
    std::cout << "Speedup Factor:            " << seqDuration.count() / parDuration.count() << "x" << std::endl;
    std::cout << "Verification (Sorted):     " << (std::is_sorted(arr2.begin(), arr2.end()) ? "PASSED" : "FAILED") << std::endl;

    return 0;
}
```

```
[Running] cd "c:\Users\Jullian Sevilla\OneDriv
"c:\Users\Jullian Sevilla\OneDrive\Documents\D
Parallel Algorithm Benchmark: Merge Sort
Dataset Size:              1000000 elements
Sequential Merge Sort Time: 951.794 ms
Parallel Merge Sort Time:   325.094 ms
Speedup Factor:            2.92775x
Verification (Sorted):     PASSED

[Done] exited with code=0 in 3.778 seconds
```

Analysis

I chose Merge Sort from the previous algorithms in the course because it splits naturally into independent sub-problems. By using threads to sort the left and right halves simultaneously, the multithreaded implementation reduced the execution time from 951.794 milliseconds down to 325.094 milliseconds. Limiting the recursion depth prevented the system from creating too many threads and incurring overhead. This parallel approach achieved a speedup factor of 2.93x while keeping the final array correctly sorted.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic

work. The following indicates the nature and extent of AI assistance:

Tools Used: **Grammarly chatgpt**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction, code assistance, and explanation of concepts.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Did not replace the original thinking**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.